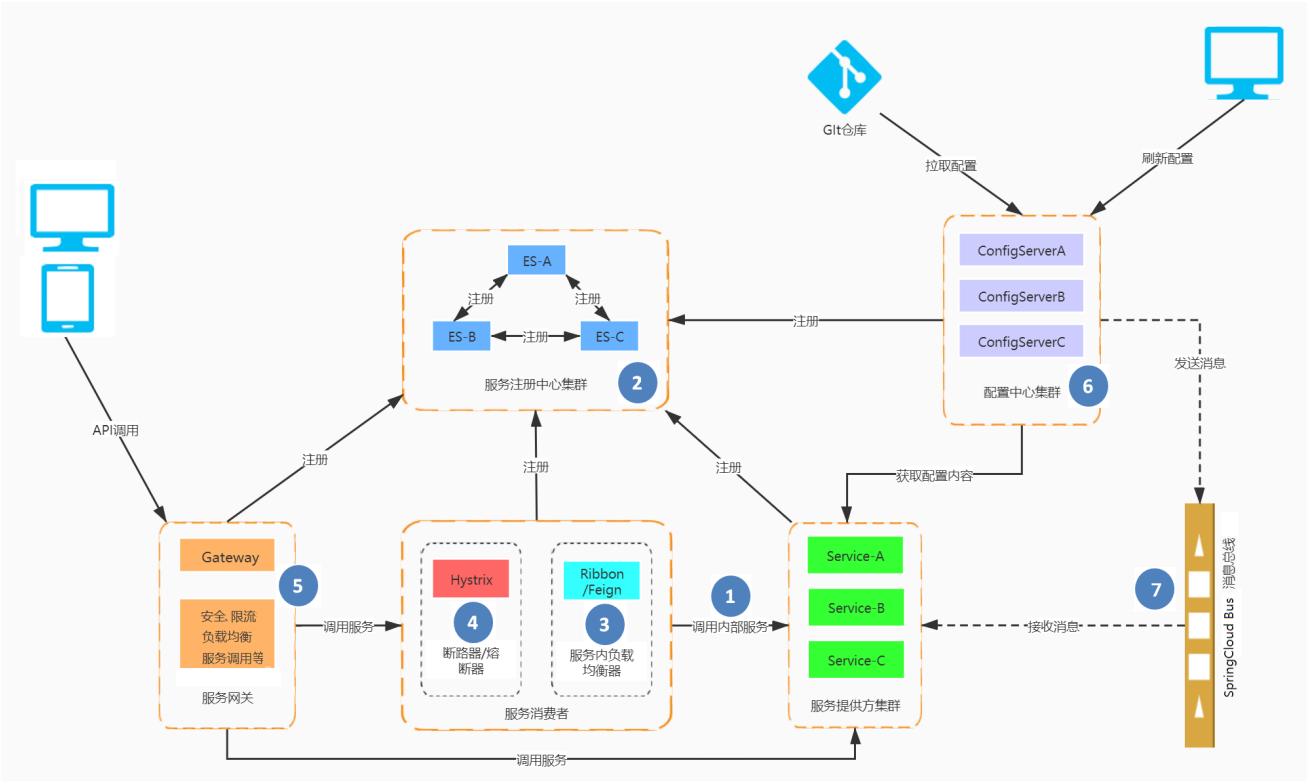
本章所讲内容为 Spring Cloud 架构图中的第五个部分。



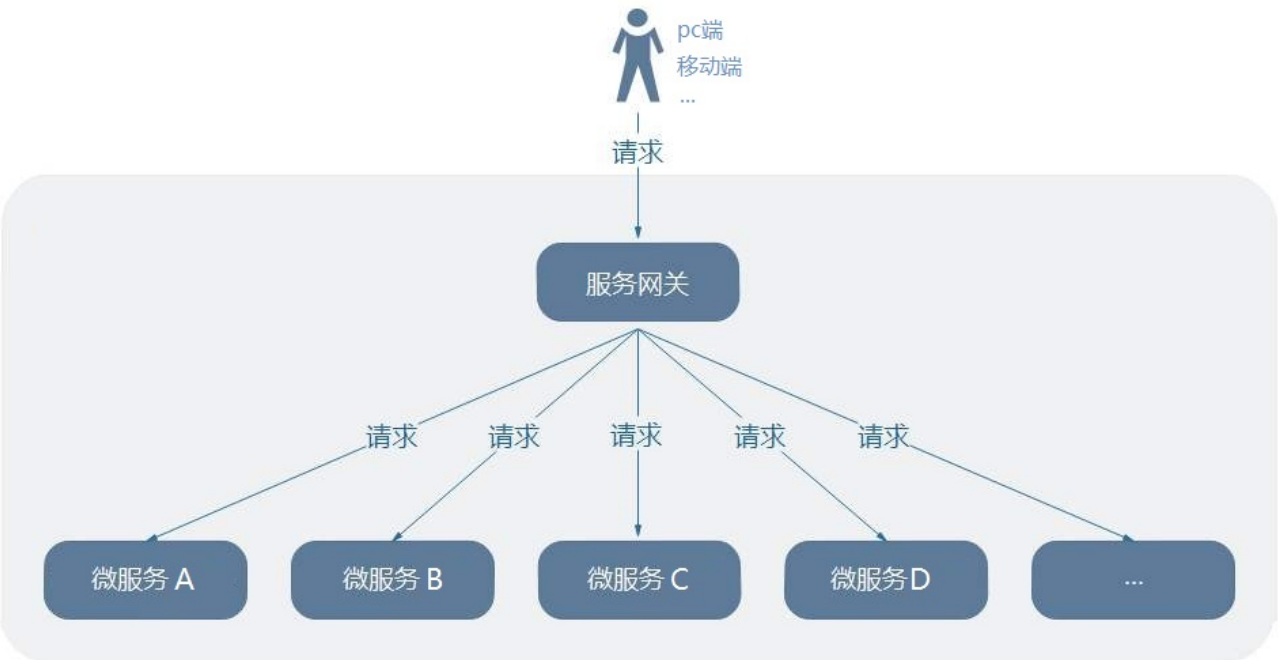
1. 微服务API网关

1.1.什么是API网关

问题的提出：访问不同的微服务API，一般会有不同的网络地址（IP和端口），客户端在访问这些微服务API时必须记住几十甚至几百个地址，这对于客户端来说太复杂也难以维护。而且，如果让客户端直接访问各个微服务API，就可能会有很多问题：

- 客户端会请求多个不同的服务，需要维护不同的请求地址，增加开发难度
- 存在跨域请求时，每个微服务都需要独立设置
- 加大身份认证的难度，每个微服务都需要独立认证
- 难以重构，随着项目的迭代，可能需要重新划分微服务。如果客户端直接与微服务通信，那么重构将会很难实施。
- 某些微服务可能使用了防火墙 / 浏览器不友好的协议，直接访问会有一定的困难。

因此，我们需要一个微服务网关，介于客户端与服务器之间的中间层，所有的外部请求都会先经过微服务网关。客户端只需要与网关交互，只知道一个网关地址即可。



综上所述，API网关 = 路由 + 过滤器

1. 路由：API网关是一个服务器，是系统对外的唯一入口，它接收所有客户端请求并转发给相应的微服务。
2. 过滤器：在网关中可以完成一系列的横切功能，例如权限校验、跨域设置、负载均衡、日志、限流以及监控等等，这些都可以通过过滤器完成（其实路由转发也是通过过滤器实现的）。

1.2.常见API网关

- Nginx：使用反向代理和负载均衡可实现对API服务器的高可用。
- Zuul：Netflix开源，使用java开发，功能丰富。
- Spring Cloud Gateway：Spring Cloud提供的新一代网关，用于替代Zuul。

2.微服务API网关Gateway

Spring Cloud Gateway 是 Spring 官方开发的网关，旨在为微服务架构提供一种简单而有效的统一的 API 路由管理方式，统一访问接口。

Spring Cloud Gateway 作为 Spring Cloud 生态系中的网关，目标是替代 Netflix的zuul1.x，并且没有对zuul2.x以上版本进行集成。Gateway不仅提供统一的路由方式，并且基于 Filter 链的方式提供了网关基本的功能，例如：路由、熔断、过滤、限流等。

2.1.Gateway核心概念

1. 路由（route） 路由是网关最基础的部分。路由信息由一个ID、一个目的URI、一组断言和一组Filter组成。如果断言为真，则说明请求URL和配置的路由匹配。

2. 断言（predicates）就是路由的条件。开发人员可以匹配HTTP请求中的所有内容（例如请求头或请求参数），如果请求与断言相匹配则进行路由。
3. 过滤器（filter） Gateway中的Filter分为两种类型，分别是Gateway Filter和Global Filter。过滤器Filter可以对请求和响应进行处理。

2.2.Gateway路由

2.2.1.路由基础案例

1. 在父工程下，创建 Maven Module 子工程（工程名：gateway_server_14000；Packaging：jar）
2. 在pom.xm文件中添加Gateway依赖

```
<dependencies>
    <!--加入gateway依赖 -->
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-gateway</artifactId>
    </dependency>

    <!--热部署 gav -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
</dependencies>
```

3. 添加主启动类

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

4. 创建application.yml 配置文件

```
server:
    port: 14000
spring:
    application:
        name: gateway-server
    #下面是关于Spring Cloud Gateway的配置
    cloud:
```

```yaml
        gateway:
            routes:
            -   id: consumerServer
                uri: http://127.0.0.1:12000
                predicates:
                -   Path=/cart/getUserById/**
            #-   id: businessServer
            #    uri: http://127.0.0.1:8887
            #    predicates:
            #    -   Path=/**
```

> 1. routes属性下可以配置多个路由，所以书写时使用 “-” 符号。
> 2. id属性：自定义路由ID，保证唯一即可。
> 3. uri属性：要访问的目标服务器地址，即Gateway的API网关地址。
> 4. predicates属性：使用断言判定路由条件。这里使用Path对请求路径进行匹配，如果为真就进行路由。
> 5. 实际上，Gateway的内置路由规则很多。除了使用Path进行路径匹配之外，还可以根据Cookie、根据Header、根据Host主机地址、根据请求方式等等进行匹配。

5. 测试：启动工程，浏览器地址栏输入：http://localhost:14000/cart/getUserById/1

> 上面案例说明：当我们访问 http://localhost:14000/cart/getUserById/1 时其实和 http://localhost:12000/cart/getUserById/1 是一样的。这就相当于不暴露具体的微服务端口（12000），而是在微服务端口外面套了一层网关端口（14000）。
>
> 这样我们在访问的时候，就可以只使用一个网关端口，却可以调用所有的微服务了。

2.2.2.动态路由（面向服务路由）

一个微服务架构中可能存在成百上千个微服务，而且这些微服务的配置很有可能会进行更新。比如：添加或移除微服务、修改微服务的IP与端口。此时，势必要重新修改网关中的路由配置。很显然，这是非常不合理的。而且，也无法实现微服务集群的负载均衡。

Gateway支持与Eureka整合开发，根据服务名自动从注册中心中获取服务地址并转发请求。这样做不仅能够通过单个端点来访问应用中的所有服务，而且在添加、移除或修改服务时，不用修改Gateway的路由配置。这就是动态路由（面向服务的路由）。

1. 在gateway_server_14000 子工程中添加Eureka Client的依赖

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

2. 在application.yml 文件中配置动态路由

```yaml
server:
    port: 14000
spring:
    application:
```

```yaml
        name: gateway-server
    #下面是关于Spring Cloud Gateway的配置
    cloud:
        gateway:
            routes:
            -   id: consumerServer
                uri: lb://consumer-server   #这里改写成服务名
                predicates:
                -   Path=/cart/getUserById/**

#eureka配置（Gateway也要称为Eureka的一个客户端）
eureka:
    client:
        service-url:
            defaultZone:
http://eurekaServer13000:13000/eureka,http://eurekaServer13001:13001/eureka
    instance:
        prefer-ip-address: true
        instance-id: ${spring.cloud.client.ip-address}:${server.port}
```

> uri以 lb: //开头（lb代表从注册中心获取服务列表），后面接的就是需要转发到的服务名。
>
> 实际上，SpringCloud提供了LoadBalancer负载均衡过滤器，它继承了ribbon的接口。它可以作用在url上，使用以lb开头的路由，通过服务名获取微服务实例，并实现了负载均衡。

3. 测试（要确保Gateway已经在Eureka上注册成功）：启动工程，浏览器地址栏输入：http://localhost:14000/cart/getUserById/1

2.3.Gateway过滤器

Spring Cloud Gateway除了具备请求路由功能之外，也支持对请求的过滤。

2.3.1.Gateway过滤器类型

Spring Cloud Gateway 的 Filter可分为两种：

- 局部过滤器（GatewayFilter接口），是针对单个路由的过滤器。
- 全局过滤器（GlobalFilter接口），作用于所有路由，不需要单独配置。开发者可以通过全局过滤器实现一些共通功能，并且全局过滤器也是开发者使用比较多的过滤器。

Spring Cloud Gateway内部也是通过一系列的内置全局过滤器对整个路由转发进行处理。

2.3.2.过滤器实例

下面创建一个统一鉴权认证过滤器，也就是对用户是否有访问权限的验证。这个鉴权过程可以在网关层统一检验，检验的标准就是：请求中是否携带token凭证以及token的正确性。

1. 在gateway_server_14000 子工程中添加filter包，创建AuthFilter类

```
package com.neusoft.filter;

import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;

//使用@Component注解将过滤器放入Spring容器中。
@Component
public class AuthFilter implements GlobalFilter {

    /**
     * 执行过滤器中的业务逻辑。
     * ServerWebExchange：相当于请求响应的上下文。
     * GatewayFilterChain：网关过滤的链表，用于过滤器的链式调用。
     */
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        //获取请求参数
        String token = exchange.getRequest().getQueryParams().getFirst("token");
        if(token==null) {
            //响应HTTP状态码（401：没有访问权限）
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            //请求结束
```

```
            return exchange.getResponse().setComplete();
        }
        //继续执行过滤器链中的下一个资源
        return chain.filter(exchange);
    }
}
```

> 注意：使用@Component注解将过滤器放入Spring容器中。

2. 测试： http://localhost:14000/cart/getUserById/1?token=1

   如果输入的地址没有?token=1，那么，会提示401错误页面。

2.4.网关熔断处理

不论是服务消费者调用服务提供者时、还是通过网关调用微服务时，都需要进行熔断处理。所以，Gateway也需要使用Hystrix进行熔断处理。也就是Gateway集成Hystrix。

在pom.xml文件中添加Hystrix依赖：

```
<!--加入hystrix的依赖 -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
</dependency>
```

在application.yml文件中添加Hystrix配置：

```
spring:
    application:
        name: gateway-server
    cloud:
        gateway:
            routes:
            -   id: businessServer
                uri: lb://consumer-server    #这里改写成服务名
                predicates:
                -   Path=/cart/getUserById/**
                filters:                     #在路由中添加Hystrix配置
                -   name: Hystrix
                    args:
                        name: fallbackcmd
                        fallbackUri: forward:/fallback
```

> - filters：配置一个Gateway过滤器。
> - filters.name：配置了name为Hystrix的filter，实际是对应HystrixGatewayFilterFactory。
> - filters.name.args.name：指定一个HystrixCommand的对象名（必须要有的一个自定义对象名）。
> - filters.name.args.fallbackUri：设置降级逻辑（注意fallbackUri要以forward开头）。

在Controller中添加降级逻辑：

```
package com.neusoft.controller;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.neusoft.po.CommonResult;

@RestController
public class FallbackController {

    @RequestMapping("/fallback")
    public CommonResult fallback(){
        //403: 发送的请求被服务器拒绝
        return new CommonResult(403,"Gateway触发了熔断降级方法",null);
    }
}
```

> application.yml 文件中配置的 forward:/fallback 对应 @RequestMapping("/fallback")

测试：

访问一个停止的（模拟宕机）微服务，将返回：

```
{"code":403,"message":"Gateway触发了熔断降级方法","result":null}
```